Embeddings & RL: A Visual Guide

Teaching Computers to Understand and Decide

Prof. Dr. Jörg Osterrieder

Methods and Algorithms — MSc Data Science

Spring 2026

How Do You Teach a Computer?

How do you teach a computer to **understand language** *and* **make decisions***?*

Today we explore two big ideas:

- **Embeddings** — understanding words
- **RL** — learning to act

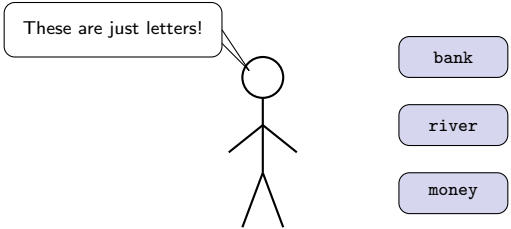
XKCD #1838 by Randall Munroe (CC BY-NC 2.5)

What You Will Learn Today

1. **Explain what word embeddings do** — in plain English
2. **Describe how an RL agent learns** from rewards
3. **Choose when to use embeddings vs RL** for a problem

Everything is explained with pictures and analogies.

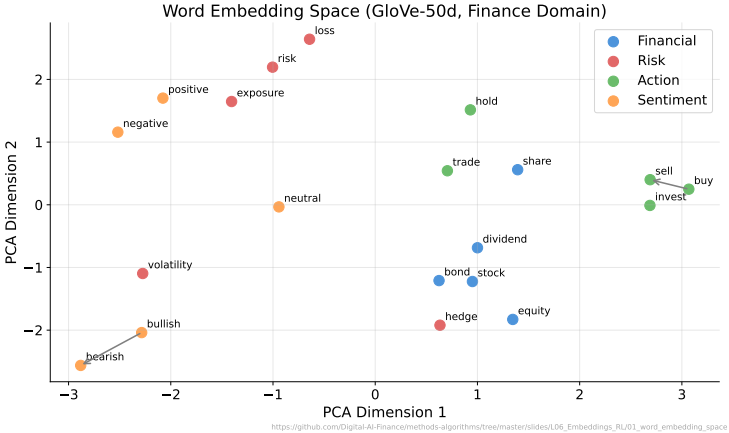
The Problem: Computers Don't Understand Words!



A computer sees text as a string of characters. It has no concept of meaning.

Computers need numbers, not words. But which numbers capture meaning?

Words Can Become Points in Space!



- Each word is placed at a specific location (coordinates)
- Similar words end up close together
- The computer learned these positions from reading text

Word embeddings turn words into vectors — lists of numbers that capture meaning.

Why Not Just Number Every Word?

Simple Numbering

- cat = 1, dog = 2, car = 3
- dog is NOT "between" cat and car
- Numbers imply an order that doesn't exist

Embeddings

- cat = [0.9, 0.1]
- dog = [0.8, 0.2]
- car = [0.1, 0.7]

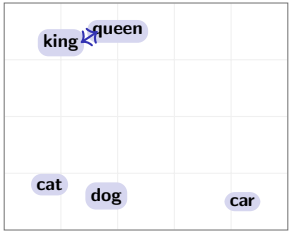
cat and dog are close (both animals). car is far away (different concept).

Key Insight

Embeddings capture relationships. Simple numbers don't.

Embeddings are learned from data — the computer discovers which words are similar by seeing them in context.

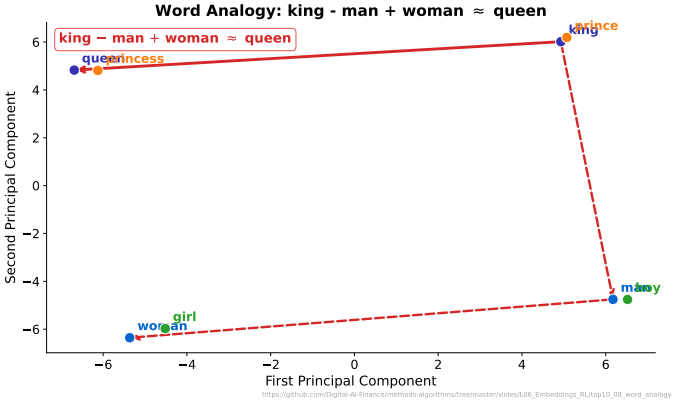
Embeddings = Giving Words a Home Address



Similar words live in the same neighborhood.

In a city, nearby addresses mean nearby locations. In embeddings, nearby vectors mean similar meanings.

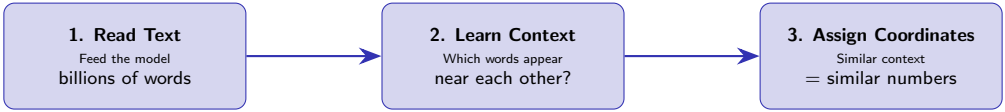
The Magic of Word Math!



- King − Man + Woman = Queen
- Paris − France + Germany = Berlin
- Embeddings capture analogies as vector arithmetic

This is the "aha moment" of embeddings: meaning has direction. Gender, country, tense — all are vectors.

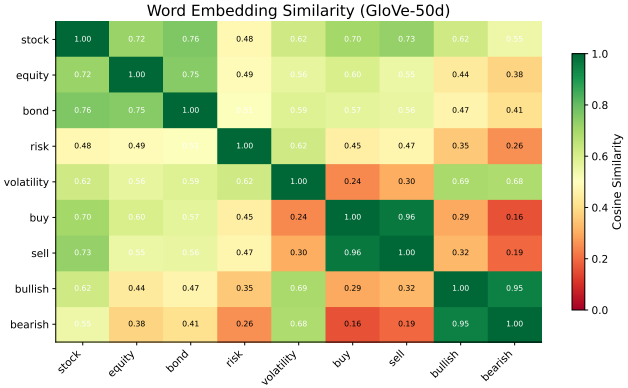
How Embeddings Are Made



That's it. The computer reads text and figures out which words are similar.

Word2Vec, GloVe, and BERT all follow this principle: learn from context.

Who Is Similar to Whom?



https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/02_similarity_heatmap

- Dark squares = highly similar words
- Light squares = unrelated words
- The computer learned this from reading text alone — no human labeled it

Cosine similarity measures the angle between two word vectors. Small angle = similar meaning.

Why Bankers Care About Embeddings

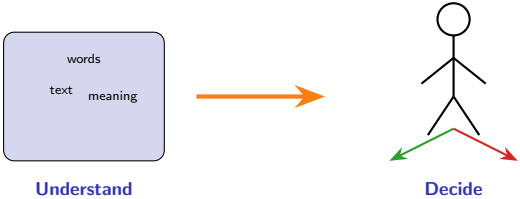
- **Sentiment Analysis**: Is this news article positive or negative for the stock?
- **Fraud Detection**: Does this transaction description look suspicious?
- **Document Search**: Find contracts similar to this one

Bottom Line

Embeddings let machines understand financial text — reports, news, filings.

JPMorgan, Goldman Sachs, and Bloomberg all use NLP with embeddings for financial analysis.

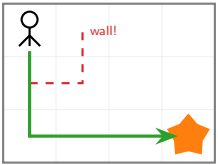
Understanding Is Powerful, But What About Action?



- Embeddings teach computers to **understand** language
- But some problems require making **sequential decisions** under uncertainty

Embeddings answer "what does this mean?" RL answers "what should I do next?"

RL: Learning by Trying, Failing, and Trying Again

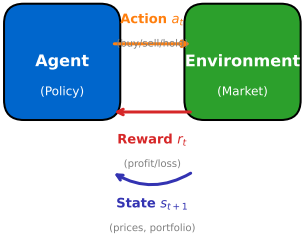


The agent doesn't know the map — it learns by exploring.

Reinforcement Learning = learn from experience, not from labeled examples.

The Core Idea: Act, Observe, Learn, Repeat

Reinforcement Learning: Agent-Environment Interaction



At each time step t:

Agent observes state, takes action, receives reward

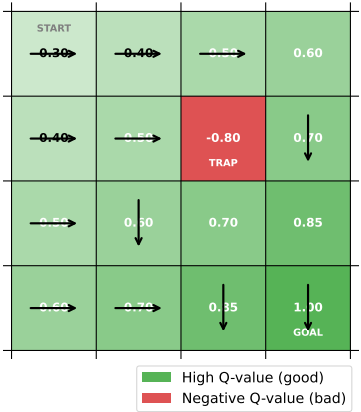
https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/03_rl_loop

- The agent takes an action in the environment
- The environment gives back a reward (or penalty)
- The agent updates its strategy and tries again

This loop is the foundation of ALL reinforcement learning. Everything else is optimization.

Watch the Agent Learn the Best Path

Q-Learning: Grid World with Learned Q-Values



https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/04_q_learning_grid

- Arrows show the agent's best guess for direction
- After many episodes, arrows converge to the optimal path
- Discovered by trial and error — no teacher told it

Q-learning stores a "quality" score for each state-action pair. Higher Q = better action.

How RL Works (No Math)

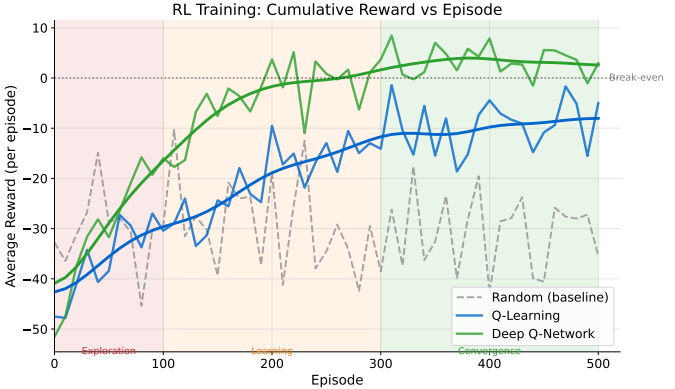
1. **Try an action** — the agent does something (move left, buy, sell) →
2. **Get feedback** — the environment gives a reward ($+1$) or penalty (-1)
3. **Update strategy** — remember what worked, avoid what didn't

The Secret

Repeat thousands of times. The agent gets better.

RL agents typically need millions of episodes to learn. That is why simulation is so important.

Proof It Works: Rewards Go Up Over Time

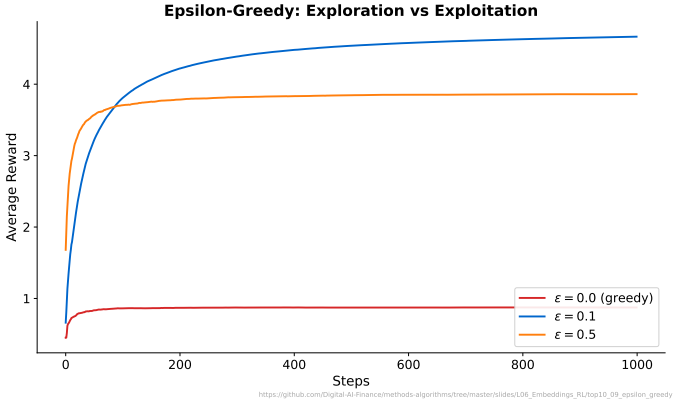


https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/05_reward_curves

- The y-axis shows cumulative reward
- The line goes up = the agent is getting better
- Early episodes are random; later episodes show learned behavior

A rising reward curve is the universal sign of a learning agent. Flat = stuck. Dropping = broken reward.

The Key Tradeoff: Explore or Exploit?



- **Explore** = try something new (might find better)
- **Exploit** = stick with what works (safe but might miss better)
- Epsilon-greedy: explore with probability epsilon, exploit otherwise

Too much exploration = never settles. Too much exploitation = misses the best strategy.

Three Things to Watch Out For

1. ! **Needs LOTS of practice** — millions of episodes. That's why we train in simulation first.
2. ! **Reward design is tricky** — the agent finds loopholes you didn't expect. Design rewards carefully.
3. ! **Sim-to-real gap** — what works in simulation may not work in the real world without fine-tuning.

RL is powerful but hungry: it needs massive compute and careful reward engineering.

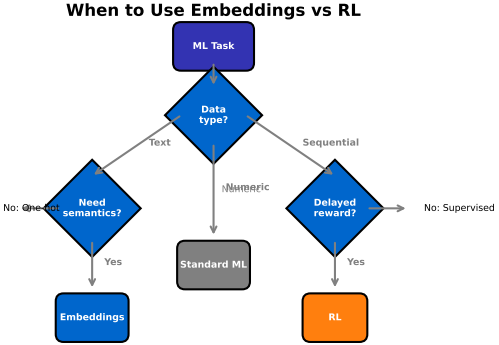
Embeddings vs RL: Quick Comparison

Aspect	Embeddings	RL
Input	Text / documents	Actions / states
Output	Number representations	Decisions / policies
Learns from	Patterns in text	Rewards and penalties
Speed	Fast (minutes)	Slow (hours to days)
Use case	Understanding	Acting

They solve different problems. Use embeddings to understand, RL to act.

In practice, you can combine them: embeddings as state representation for RL agents.

Which Method Should I Use?



https://github.com/Digital-AI-Finance/methods-algorithms/tree/master/slides/L06_Embeddings_RL/07_decision_flowchart

- Start at the top and follow the arrows
- In practice, many real systems combine both

When in doubt: embeddings for text understanding, RL for sequential decision-making.

AI in Finance: Embeddings + RL

Embeddings in Finance

- Chatbots for customer service
- Credit scoring from application text
- ESG report analysis at scale

RL in Finance

- Algorithmic trading strategies
- Portfolio rebalancing
- Optimal order execution

The biggest banks and hedge funds use both: NLP for understanding markets, RL for acting on them.

Try It Yourself

Embeddings (3 lines)

```
from gensim.models import Word2Vec
model = Word2Vec(sentences,
                 vector_size=100)
similar = model.wv.most_similar("bank")
```

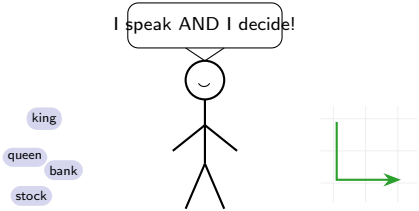
RL (plain English)

```
for each episode:
  observe state, pick action
  get reward, update strategy
  repeat until done
```

Embeddings: 3 lines of real Python. RL: the algorithm in plain English.

For embeddings: pip install gensim. For RL: start with OpenAI Gym environments.

From Confusion to Intelligence



Embeddings teach understanding. RL teaches action. Together: modern AI.

Embeddings teach understanding. RL teaches action. Together, they are the foundation of modern AI.

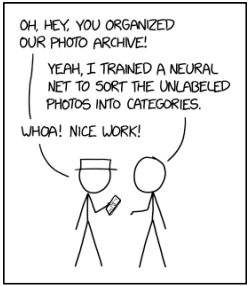
Five Things to Remember

1. Embeddings turn words into numbers that capture meaning
2. Similar words get similar numbers — the computer learns this from text
3. RL agents learn by trial and error with rewards
4. Explore vs exploit is the key RL tradeoff
5. Both are used in finance: embeddings for text, RL for decisions

---

Embeddings for understanding. RL for acting. Use both.

Go Teach a Machine Something



ENGINEERING TIP:
WHEN YOU DO A TASK BY HAND, YOU CAN TECHNICALLY SAY YOU TRAINED A NEURAL NET TO DO IT.

XKCD #2173 by Randall Munroe (CC BY-NC 2.5). Next: try the Jupyter notebook with real financial data!